

```
import pandas as pd
from ISLP import load_data
import matplotlib.pyplot as plt
import seaborn as sns
```

(a) To begin, load in the Boston data set, which is part of the ISLP library.

```
# Load the Boston dataset using ISLP
```

```
Boston = load_data('Boston')
```

```
# Save the DataFrame to a CSV file
```

```
Boston.to_csv('Boston.csv', index=False) # index=False prevents writing the DataFrame index to the CSV
```

```
Boston = pd.read_csv('Boston.csv')
```

(b) How many rows are in this data set? How many columns? What do the rows and columns represent? There are 506 rows and 13 columns, representing 506 data points and 13 variables, respectively.

Boston

c) Make some pairwise scatterplots of the predictors (columns) in this data set. Describe your findings.

There is a positive relationship between Median value of owner-occupied homes in 1000's and Percentage of lower status of the population.

(d) Are any of the predictors associated with per capita crime rate? If so, explain the relationship.

This section lists the variables with a strong correlation with crim, where strong is defined as having an absolute correlation coefficient greater than 0.5. Absolute value is used to consider both strong positive and strong negative correlations.

Results: crim 1.000000 rad 0.625505 tax 0.582764 This indicates that rad and tax have a relatively strong linear relationship with crim.

In Summary: This output indicates that the index of accessibility to radial highways (rad) and the property tax rate (tax) have the strongest positive correlations with per capita crime rate (crim). While other variables also show some correlation, the strength is weaker. Remember that correlation does not imply causation. These results only

show that there is a statistical association between these variables and crime rate, but it does not prove that one variable causes changes in the other.

```
# Calculate correlations with 'crim'
```

```
correlations =
```

```
Boston.corr()['crim'].sort_values(ascending=False)
```

```
# Print the correlations
```

```
print("Correlations with per capita crime rate (crim):")
```

```
print(correlations)
```

```
# Identify potentially strong associations (threshold of 0.5 for demonstration)
```

```
strong_associations = correlations[abs(correlations) > 0.5]
```

```
print("\nPotentially strong associations (absolute correlation > 0.5):")
```

```
print(strong_associations)
```

```
Correlations with per capita crime rate (crim):
```

```
crim    1.000000
```

```
rad     0.625505
```

```
tax     0.582764
```

```
lstat   0.455621
```

```
nox     0.420972
```

```
indus   0.406583
```

```
age     0.352734
```

```
prratio 0.289946
```

```
chas    -0.055892
```

```
zn      -0.200469
```

```
rm      -0.219247
```

```
dis     -0.379670
```

```
medv    -0.388305
```

```
Name: crim, dtype: float64
```

```
Potentially strong associations (absolute correlation > 0.5):
```

```
crim    1.000000
```

```
rad     0.625505
```

```
tax     0.582764
```

```
Name: crim, dtype: float64
```

(e) Do any of the suburbs of Boston appear to have particularly high crime rates? Tax rates? Pupil-teacher ratios? Comment on the range of each predictor.

Suburbs with High Crime Rates (above 90th percentile

-High industrialization rate. -nox, 0.67, which is slightly high but significant. The maximum in age does not necessarily correlate with medium. The age however is around 93%, meaning those properties are old and not well funded.

2. All the tax rates are at the highest at 666, meaning those don't get a lot of tax relief and could contribute to the lower prices on those properties. The values range from low- to high end on these data.

3. Suburbs with High Pupil-Teacher Ratios (above 90th percentile): It appears the ratios have high pupil populations which correlates well with crime. With 56 values, or high-PTRATIO the property values are extremely low. What we know is that there are many low level value property, probably related to high industrialisation rate properties, high concentrations, pollution, without a zoning code.

```
# Print the lists of suburbs with high values
# Calculate descriptive statistics of those suburbs
print("\nSuburbs with high crime rates (above 90th
percentile):", high_crim_suburbs.index.tolist())
print(high_crim_suburbs.describe())

print("\nSuburbs with high tax rates (above 90th
percentile):", high_tax_suburbs.index.tolist())
print(high_tax_suburbs.describe())

print("\nSuburbs with high pupil-teacher ratios (above 90th
percentile):", high_ptratio_suburbs.index.tolist())
print(high_ptratio_suburbs.describe())

# Calculate and print the range of each predictor
predictor_ranges = Boston[['crim', 'tax', 'ptratio']].agg(['min',
'max'])
print("\nRange of each predictor:\n", predictor_ranges)
Descriptive statistics for crim, tax, and ptratio:
      crim      tax  ptratio
count 506.000000 506.000000 506.000000
mean   3.613524 408.237154 18.455534
std    8.601545 168.537116  2.164946
min    0.006320 187.000000 12.600000
25%    0.082045 279.000000 17.400000
50%    0.256510 330.000000 19.050000
75%    3.677083 666.000000 20.200000
```

max 88.976200 711.000000 22.000000

Suburbs with high crime rates (above 90th percentile): [367, 373, 374, 375, 376, 378, 379, 380, 381, 384, 385, 386, 387, 388, 392, 394, 398, 400, 401, 403, 404, 405, 406, 407, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 422, 425, 426, 427, 434, 435, 436, 437, 438, 440, 444, 468, 469, 477, 479]

	crim	zn	indus	chas	nox	rm	age
count	51.000000	51.0	5.100000e+01	51.0	51.000000	51.000000	51.000000
mean	22.652214	0.0	1.810000e+01	0.0	0.672333	5.750373	93.743137
std	16.588706	0.0	1.076419e-14	0.0	0.046287	0.803842	9.779064
min	10.834200	0.0	1.810000e+01	0.0	0.580000	3.863000	56.700000
25%	13.600150	0.0	1.810000e+01	0.0	0.659000	5.290500	91.550000
50%	15.860300	0.0	1.810000e+01	0.0	0.679000	5.854000	97.300000
75%	24.021000	0.0	1.810000e+01	0.0	0.700000	6.361500	100.000000
max	88.976200	0.0	1.810000e+01	0.0	0.740000	7.313000	100.000000

	dis	rad	tax	ptratio	lstat	medv
count	51.000000	51.0	51.0	5.100000e+01	51.000000	51.000000
mean	1.693161	24.0	666.0	2.020000e+01	22.921961	12.272549
std	0.351769	0.0	0.0	1.794032e-14	6.943948	5.355822
min	1.137000	24.0	666.0	2.020000e+01	10.110000	5.000000
25%	1.452500	24.0	666.0	2.020000e+01	17.630000	8.450000
50%	1.607400	24.0	666.0	2.020000e+01	22.980000	10.500000
75%	1.864550	24.0	666.0	2.020000e+01	27.075000	14.450000
max	2.908400	24.0	666.0	2.020000e+01	37.970000	27.900000

Suburbs with high tax rates (above 90th percentile): [356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437, 438, 439, 440, 441, 442, 443, 444, 445, 446, 447, 448, 449, 450, 451, 452, 453, 454, 455, 456, 457, 458, 459, 460, 461, 462, 463, 464, 465, 466, 467, 468, 469, 470, 471, 472, 473, 474, 475, 476, 477, 478, 479, 480, 481, 482, 483, 484, 485, 486, 487, 488, 489, 490, 491, 492]

	crim	zn	indus	chas	nox	rm \
count	137.000000	137.0	137.000000	137.000000	137.000000	137.000000
mean	12.299162	0.0	18.451825	0.058394	0.670102	6.006212
std	13.017296	0.0	1.814342	0.235348	0.061051	0.713021
min	0.105740	0.0	18.100000	0.000000	0.532000	3.561000
25%	5.441140	0.0	18.100000	0.000000	0.614000	5.683000
50%	8.716750	0.0	18.100000	0.000000	0.693000	6.152000
75%	14.236200	0.0	18.100000	0.000000	0.713000	6.411000
max	88.976200	0.0	27.740000	1.000000	0.770000	8.780000

	age	dis	rad	tax	ptratio	lstat \
count	137.000000	137.000000	137.000000	137.000000	137.000000	137.000000
mean	89.967883	2.05447	23.270073	667.642336	20.196350	18.674526
std	12.469840	0.60790	3.764196	8.469441	0.018821	6.873577
min	40.300000	1.12960	4.000000	666.000000	20.100000	2.960000
25%	86.100000	1.58950	24.000000	666.000000	20.200000	14.190000
50%	94.500000	1.92650	24.000000	666.000000	20.200000	18.050000
75%	98.800000	2.36820	24.000000	666.000000	20.200000	23.240000

max 100.000000 4.09830 24.000000 711.000000 20.2
00000 37.970000

medv
count 137.000000
mean 16.272263
std 8.459008
min 5.000000
25% 11.000000
50% 14.300000
75% 19.900000
max 50.000000

Suburbs with high pupil-teacher ratios (above 90th percentil
e): [13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 2
8, 29, 30, 31, 32, 33, 34, 54, 100, 101, 102, 103, 104, 105, 10
6, 107, 108, 109, 110, 127, 128, 129, 130, 131, 132, 133, 13
4, 135, 136, 137, 138, 139, 140, 141, 354, 355, 501, 502, 50
3, 504, 505]

crim zn indus chas nox rm \
count 56.000000 56.000000 56.000000 56.0 56.000000 5
6.000000
mean 0.577256 4.196429 11.947500 0.0 0.553875 6.0
69964
std 0.453506 17.806454 6.313951 0.0 0.052941 0.37
1661
min 0.013600 0.000000 1.910000 0.0 0.410000 5.01
9000
25% 0.146395 0.000000 8.140000 0.0 0.533500 5.8
31000
50% 0.551150 0.000000 8.560000 0.0 0.538000 6.0
38500
75% 0.899880 0.000000 21.890000 0.0 0.624000 6.3
28250
max 1.628640 80.000000 21.890000 0.0 0.624000 6.
976000

age dis rad tax ptratio lstat \
count 56.000000 56.000000 56.000000 56.000000 56.00
0000 56.000000
mean 83.776786 3.392082 3.910714 357.767857 21.07
1429 14.520000
std 20.043054 1.819077 1.014025 61.476970 0.21038
0 5.523465
min 19.500000 1.439400 1.000000 273.000000 20.900
000 5.570000

```

25%  81.475000  2.200125  4.000000  307.000000  21.00
0000 10.997500
50%  91.100000  2.746250  4.000000  334.000000  21.00
0000 14.340000
75%  96.950000  4.234500  4.000000  437.000000  21.20
0000 17.287500
max  100.000000 10.585700  5.000000  469.000000  22.0
00000 34.410000

```

```

      medv
count 56.000000
mean  18.123214
std   3.456903
min   11.900000
25%   15.100000
50%   18.300000
75%   20.125000
max   27.500000

```

Range of each predictor:

```

      crim tax ptratio
min  0.00632 187    12.6
max  88.97620 711    22.0

```

(f) How many of the suburbs in this data set bound the Charles river? 35

```

print(Boston['chas'].value_counts()) #如果區域接壤河流則
為 1, 否則為 0

```

```

chas
0    471
1     35

```

Name: count, dtype: int64

(g) What is the median pupil-teacher ratio among the towns in this data set? The median pupil-teacher ratio is: 19.05

(h) Which suburb of Boston has lowest median value of owner- occupied homes? What are the values of the other predictors for that suburb, and how do those values compare to the overall ranges for those predictors? Comment on your findings.

Based on this analysis, the suburb with the lowest median home value exhibits a combination of negative characteristics: High Crime Rate: A significant deterrent to home values. High Industrialization and Pollution: Undesirable environmental factors. Old Housing Stock:

Potentially less desirable and requiring more maintenance. High Property Taxes: Making homeownership more expensive. High Pupil-Teacher Ratio: Possibly indicating under-resourced schools. High Proportion of Lower-Status Population: Potentially reflecting socioeconomic challenges. Good Access to Highways and Employment Does not outweigh all the other negative factors. This paints a picture of a suburb that faces multiple challenges, contributing to its low median home value. It's important to remember this is just an association, and we can't definitively say that these factors cause the low home values, but they certainly contribute to a less desirable living environment. It would be important to do further investigation to see which of those things are the most important.

Find the suburb with the lowest median value

```
min_medv_index = Boston['medv'].idxmin()
lowest_medv_suburb = Boston.loc[min_medv_index]
```

Print the suburb and its 'medv' value

```
print(f'Suburb with the lowest median value:
{min_medv_index}')
print(f'Lowest median value (medv):
{lowest_medv_suburb['medv']}')
```

Print the other predictor values for that suburb

```
print("\nOther predictor values for this suburb:")
print(lowest_medv_suburb.drop('medv')) # Drop 'medv' so
we only see the predictors
```

Calculate the overall ranges for the predictors

```
predictor_ranges = Boston.drop('medv', axis=1).agg(['min',
'max']) # Drop 'medv' before getting the ranges
```

Print the predictor ranges

```
print("\nOverall ranges for the predictors:")
print(predictor_ranges)
```

*# Compare the suburb's values to the overall ranges and
comment*

```
print("\nComparison and Findings:")
```

Print comparison statement for each predictor

```
for predictor in predictor_ranges.columns:
    suburb_value = lowest_medv_suburb[predictor]
    min_value = predictor_ranges[predictor]['min']
```



```

max_value = predictor_ranges[predictor]['max']

print(f" {predictor}: Suburb value = {suburb_value},
Range = ({min_value}, {max_value})")
Suburb with the lowest median value: 398
Lowest median value (medv): 5.0

```

Other predictor values for this suburb:

```

crim    38.3518
zn      0.0000
indus   18.1000
chas    0.0000
nox     0.6930
rm      5.4530
age     100.0000
dis     1.4896
rad     24.0000
tax     666.0000
ptratio 20.2000
lstat   30.5900

```

Name: 398, dtype: float64

Overall ranges for the predictors:

```

      crim  zn indus chas  nox  rm  age  dis rad tax
\
min 0.00632  0.0  0.46   0 0.385 3.561  2.9 1.1296
1 187
max 88.97620 100.0 27.74   1 0.871 8.780 100.0 12.12
65 24 711

```

```

      ptratio lstat
min   12.6   1.73
max   22.0  37.97

```

Comparison and Findings:

```

crim: Suburb value = 38.3518, Range = (0.00632, 88.9762)
zn: Suburb value = 0.0, Range = (0.0, 100.0)
indus: Suburb value = 18.1, Range = (0.46, 27.74)
chas: Suburb value = 0.0, Range = (0, 1)
nox: Suburb value = 0.693, Range = (0.385, 0.871)
rm: Suburb value = 5.453, Range = (3.561, 8.78)
age: Suburb value = 100.0, Range = (2.9, 100.0)
dis: Suburb value = 1.4896, Range = (1.1296, 12.1265)
rad: Suburb value = 24.0, Range = (1, 24)
tax: Suburb value = 666.0, Range = (187, 711)
ptratio: Suburb value = 20.2, Range = (12.6, 22.0)

```

```
lstat: Suburb value = 30.59, Range = (1.73, 37.97)
# Count suburbs with more than 7 rooms
more_than_7_rooms = Boston[Boston['rm'] > 7].shape[0]

# Count suburbs with more than 8 rooms
more_than_8_rooms = Boston[Boston['rm'] > 8].shape[0]
```